CSN3001 - Data Structures Lab
Introduction to Recursion

1. Objectives & General Guidelines

Recursion is a programming technique where a function calls itself to solve a smaller instance of the same problem. Every recursive function must possess two core components:

- **Base Case:** The terminating condition that stops recursion without making further recursive calls.
- **Recursive Step:** The logic that breaks the problem into a smaller sub-problem and invokes the function recursively.

2. Lab Problems

Problem 1: Factorial of a Non-Negative Integer

Write a recursive function factorial(n) that takes a non-negative integer n and returns its factorial.
The factorial of n (denoted as n!) is the product of all positive integers less than or equal to n.

Recursive Mathematical Definition:

```
Base Case: 0! = 1
Recursive Case: n! = n × (n - 1)!  for n > 0
```

Sample Execution & Input/Output:

```
Input: n = 5
Output: 120 (Explanation: 5 × 4 × 3 × 2 × 1 = 120)
```

Constraints & Requirements: *0 ≤ n ≤ 12. Assume input fits within standard 32-bit integer limits.*

Problem 2: N-th Fibonacci Number

Write a recursive function fibonacci(n) that returns the n-th number in the Fibonacci sequence.
The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones, starting from 0 and 1.

Recursive Mathematical Definition:

```
Base Cases: Fib(0) = 0, Fib(1) = 1
Recursive Case: Fib(n) = Fib(n - 1) + Fib(n - 2)  for n ≥ 2
```

Sample Execution & Input/Output:

```
Input: n = 7
Output: 13 (Sequence: 0, 1, 1, 2, 3, 5, 8, 13)
```

Constraints & Requirements: *0 ≤ n ≤ 30. Note the growth rate of recursive calls and call stack depth.*

Problem 3: Sum of Digits of a Positive Integer

Write a recursive function sum_of_digits(n) that accepts a positive integer n and returns the sum of its individual digits.
Do NOT convert the integer to a string; use modulo (%) and integer division (//) arithmetic operations.

Recursive Mathematical Definition:

```
Base Case: If n < 10, return n
Recursive Case: sum_of_digits(n) = (n % 10) + sum_of_digits(n // 10)
```

Sample Execution & Input/Output:

```
Input: n = 1234
Output: 10 (Explanation: 1 + 2 + 3 + 4 = 10)
```

Constraints & Requirements: *1 ≤ n ≤ 10^9. Ensure no iterative loops (while/for) are used.*

Problem 4: Recursive String Reversal & Palindrome Check

Write a recursive function is_palindrome(s) that checks whether a given string s is a palindrome (reads the same forward and backward).
Your function should ignore case and non-alphanumeric characters or process a sanitized alphanumeric string recursively.

Recursive Mathematical Definition:

```
Base Cases: If length(s) ≤ 1, return True.
Recursive Case: If first_char == last_char, return
is_palindrome(sub_string_without_ends); else return False.
```

Sample Execution & Input/Output:

```
Input: s = "radar"
Output: True
Input: s = "recursion"
Output: False
```

Constraints & Requirements: *Length of string s ≤ 100. Must be solved purely recursively without using string slicing tricks like s[::-1].*